JOB TRACKING SYSTEM

Full Stack Web Application

Complete Project Documentation Report

TABLE OF CONTENTS

Abstract
Introduction
Problem Statement
Objectives
Scope of the Project
Technologies Used
System Architecture
Detailed Module Description
API Design
Database Design
System Workflow
Features
Advantages
Limitations
Future Enhancements
Conclusion

1. ABSTRACT

The **Job Tracking System** is a comprehensive full-stack web application designed to streamline the job search and recruitment process for multiple user types including job seekers, recruiters, and system administrators. Built using modern web technologies including React.js for the frontend, Node.js with Express for the backend, and MongoDB Atlas as the cloud database, this system provides a unified platform for managing job applications, creating professional resumes, posting job vacancies, and tracking application statuses in real-time.

The application addresses the common challenges faced during job hunting by offering features such as a visual dashboard with analytics, a resume builder with multiple sections, a job portal connecting seekers with recruiters, skill extraction from job descriptions, and comprehensive administrative controls. The system enables users to organize their job search efficiently, maintain detailed records of applications, receive deadline reminders, and generate insights through data visualization.

This documentation provides a complete technical overview of the system architecture, implementation details, module descriptions, API specifications, database schema, and deployment considerations. The project demonstrates the practical application of full-stack development principles, RESTful API design, NoSQL database management, and modern frontend frameworks in solving real-world employment-related challenges.

Keywords: Job Tracking, Resume Builder, Recruitment Portal, MERN Stack, Analytics Dashboard, Full-Stack Web Application

2. INTRODUCTION

2.1 Background

In today's competitive job market, managing multiple job applications across various companies has become increasingly complex. Job seekers often struggle to keep track of application deadlines, interview schedules, and follow-up dates. Similarly, recruiters face challenges in managing job postings, reviewing applications, and tracking candidate progress through the hiring pipeline. Traditional methods using spreadsheets or manual notes are inefficient and prone to errors.

2.2 Project Overview

The Job Tracking System is designed as a centralized platform that bridges the gap between job seekers and recruiters. It provides specialized tools for different user roles:

- **Job Seekers** can track their applications, build professional resumes, apply for jobs, and monitor their application status.
- **Recruiters** can post job openings, review applicants, manage hiring pipelines, and access analytics on their job postings.
- **Administrators** can oversee system operations, manage user accounts, and maintain platform integrity.

2.3 Project Motivation

The motivation behind this project stems from the need for:

Organization - Centralized tracking of all job-related activities

Efficiency - Automated reminders and status updates

Professionalism - Well-structured resume creation tools

Connectivity - Direct communication channel between seekers and recruiters

Insights - Data-driven understanding of job search progress

2.4 Development Approach

The project follows a modular development approach with:

- Separation of concerns between frontend and backend
- RESTful API architecture for communication
- NoSQL database for flexible data storage
- Component-based frontend architecture
- Role-based access control for security

3. PROBLEM STATEMENT

3.1 Current Challenges

For Job Seekers:

Disorganized Job Search: Managing numerous applications across different companies using scattered notes or spreadsheets

Missed Deadlines: Forgetting application deadlines, interview dates, and follow-up schedules

Resume Management: Difficulty in creating, updating, and tailoring resumes for different positions

Status Tracking: Lack of visibility into application progress and recruiter responses

Skill Gap Analysis: No automated way to identify required skills from job descriptions

For Recruiters:

Application Overload: Managing large volumes of applications manually

Communication Gaps: Delayed or inconsistent communication with candidates

Pipeline Visibility: Limited insights into hiring funnel and conversion rates

Document Management: Handling resumes and cover letters efficiently

Decision Tracking: Maintaining records of acceptance/rejection decisions

For System Administrators:

User Management: Difficulty in managing different user types and permissions

Data Integrity: Ensuring data consistency and security across the platform

Platform Monitoring: Lack of tools to oversee system usage and performance

3.2 Proposed Solution

The Job Tracking System addresses these challenges by providing:

Centralized Platform: Single dashboard for all job search activities

Automated Tracking: Date tracking with visual calendar for deadlines

Resume Builder: Structured form-based resume creation with multiple sections

Real-time Updates: Instant status updates on applications

Skill Analysis: Automated skill extraction from job descriptions

Recruiter Tools: Dedicated dashboard for job posting and applicant management

Analytics: Visual charts showing application statistics and hiring metrics

4. OBJECTIVES

4.1 Primary Objectives

Develop a User-Friendly Interface: Create an intuitive, responsive web interface accessible across devices

Implement Multi-Role Support: Build distinct functionalities for job seekers, recruiters, and administrators

Enable Job Application Management: Allow users to add, edit, delete, and track job applications

Create Resume Building Tools: Provide a comprehensive resume builder with multiple customizable sections

Establish Job Portal: Connect job seekers with recruiters through a dedicated portal

Implement Analytics Dashboard: Display visual statistics using charts and graphs

Ensure Data Security: Implement proper authentication, authorization, and data validation

4.2 Secondary Objectives

Skill Extraction: Develop a keyword-based skill extraction system from job descriptions

File Upload Support: Enable resume and cover letter uploads with proper storage

Deadline Management: Implement calendar-based deadline tracking with visual indicators

Search Functionality: Provide company search capabilities for research

Mobile Responsiveness: Ensure the application works seamlessly on mobile devices

Performance Optimization: Optimize database queries and API response times

Scalable Architecture: Design the system to handle growing user base and data volume

4.3 Technical Objectives

Full-Stack Implementation: Build both frontend and backend with modern technologies
RESTful API Design: Create well-structured, documented API endpoints
NoSQL Database: Implement MongoDB for flexible, schema-less data storage
Cloud Deployment: Deploy database on MongoDB Atlas for reliability and scalability
Component-Based Frontend: Use React components for modular, reusable UI elements
Version Control: Maintain code using Git for collaboration and tracking

5. SCOPE OF THE PROJECT

5.1 In-Scope Features

Core Features:

User registration and authentication with role-based access
Job application tracking (CRUD operations)
Resume builder with multiple sections (personal info, experience, education, skills, projects)
Job portal with job posting and application functionality
Admin dashboard for user and job management
Analytics and reporting with visual charts
Skill extraction from job descriptions
File upload for resumes and cover letters
Deadline calendar with visual indicators
Company search functionality

User Roles:

Job Seeker/Client: Track applications, build resumes, apply for jobs
Recruiter: Post jobs, review applications, manage hiring pipeline
Administrator: Manage users, oversee system, delete inappropriate content

5.2 Out-of-Scope Features

Real-time messaging/chat between users and recruiters
Email notification system for updates and reminders
Payment gateway integration for premium features
AI-based resume scoring or matching algorithms
Third-party job board integrations (Indeed, LinkedIn, etc.)
Video interview scheduling and recording
Background check integration
Mobile native application (iOS/Android)
Multi-language support
Advanced AI-based skill matching using machine learning models

5.3 Target Users

Fresh Graduates: Students entering the job market seeking their first employment
Experienced Professionals: Individuals looking to switch jobs or advance careers
Recruiters: HR professionals and hiring managers seeking candidates
Career Counselors: Professionals assisting clients with job searches
Educational Institutions: Career services departments helping students

5.4 Expected Outcomes

Reduced time spent on job application organization
Improved application-to-interview conversion rates
Better resume quality through structured building tools
Enhanced visibility into hiring pipeline for recruiters
Data-driven insights for job search strategy optimization
Centralized platform reducing the need for multiple tools

6. TECHNOLOGIES USED

6.1 Frontend Technologies

React.js (v19.2.4)

React is a JavaScript library for building user interfaces. It enables the creation of reusable UI components and manages application state efficiently.

Key Features Used:

- Component-based architecture for modular development
- JSX syntax for declarative UI descriptions
- Virtual DOM for optimized rendering performance
- React Hooks for state management in functional components
- Props and state for data flow management

Why React:

- Large ecosystem and community support
- Efficient rendering with Virtual DOM
- Reusable component library
- Strong developer tools and debugging support

React Router DOM (v7.13.1)

A routing library for React applications enabling navigation between different pages.

Features:

- Declarative routing definitions
- Route parameters and query strings
- Nested routing support
- Programmatic navigation
- Protected route implementation

Chart.js (v4.5.1) and Recharts (v3.8.0)

Libraries for creating interactive data visualizations.

Applications:

- Application status distribution charts
- Job posting analytics
- Timeline visualizations
- Statistics dashboards

Axios (v1.13.6)

Promise-based HTTP client for making API requests.

Features:

- Request and response interceptors
- Automatic JSON parsing
- Request timeout configuration
- Error handling capabilities

CSS3 and Responsive Design

Custom styling with App.css and index.css for:

- Responsive layouts using Flexbox and Grid
- Mobile-first design approach
- Consistent color scheme and typography
- Animation and transition effects

6.2 Backend Technologies

Node.js

JavaScript runtime environment that executes JavaScript code outside a web browser.

Advantages:

- Single language across full stack (JavaScript)
- Non-blocking I/O for high concurrency
- Large npm package ecosystem
- Efficient for I/O-intensive applications

Express.js (v4.22.1)

Fast, unopinionated, minimalist web framework for Node.js.

Features Used:

- Middleware architecture for request processing
- Route definitions for API endpoints
- Request body parsing (JSON and URL-encoded)
- CORS (Cross-Origin Resource Sharing) handling
- Static file serving capabilities

Middleware Used:

cors (v2.8.6): Enable cross-origin requests from frontend
express.json(): Parse JSON request bodies (10MB limit)
express.urlencoded(): Parse URL-encoded data (10MB limit)

6.3 Database Technology

MongoDB Atlas

Cloud-hosted NoSQL database service providing MongoDB as a service.

Characteristics:

- Document-oriented storage (JSON-like BSON format)

- Schema-less design for flexibility
- Horizontal scalability through sharding
- High availability with replica sets
- Automatic backups and monitoring

Database Driver:

- **MongoDB Node.js Driver (v6.21.0)**: Official driver for Node.js applications

Collections Overview:

users: User authentication and profile data
jobs: Job application records
resumes: Resume builder data
job_postings: Recruiter job listings
job_applications: Job application submissions with attachments
job_seeker_resumes: Uploaded resume files

6.4 Development Tools

Environment Management

- **dotenv (v16.6.1)**: Load environment variables from .env files
- Environment variables for:
- MongoDB connection URI
- Server port configuration
- Database name

Development Utilities

- **nodemon (v3.0.1)**: Auto-restart server on file changes (development only)
- **concurrently (v8.2.2)**: Run multiple npm scripts simultaneously

Version Control

- **Git**: Distributed version control system

- **.gitignore**: Exclude node_modules and environment files

6.5 Architecture Patterns

RESTful API Design

- Resource-based URL patterns
- HTTP method semantics (GET, POST, PUT, DELETE, PATCH)
- JSON data format for requests and responses
- Status code conventions (200, 201, 400, 401, 403, 404, 500)

MVC Pattern Variation

- **Models**: MongoDB collections and schemas
- **Views**: React components (frontend)
- **Controllers**: Express route handlers

Security Measures

- CORS configuration for controlled access
- Input validation on all endpoints
- Role-based authorization checks
- ObjectId validation before database operations

7. SYSTEM ARCHITECTURE

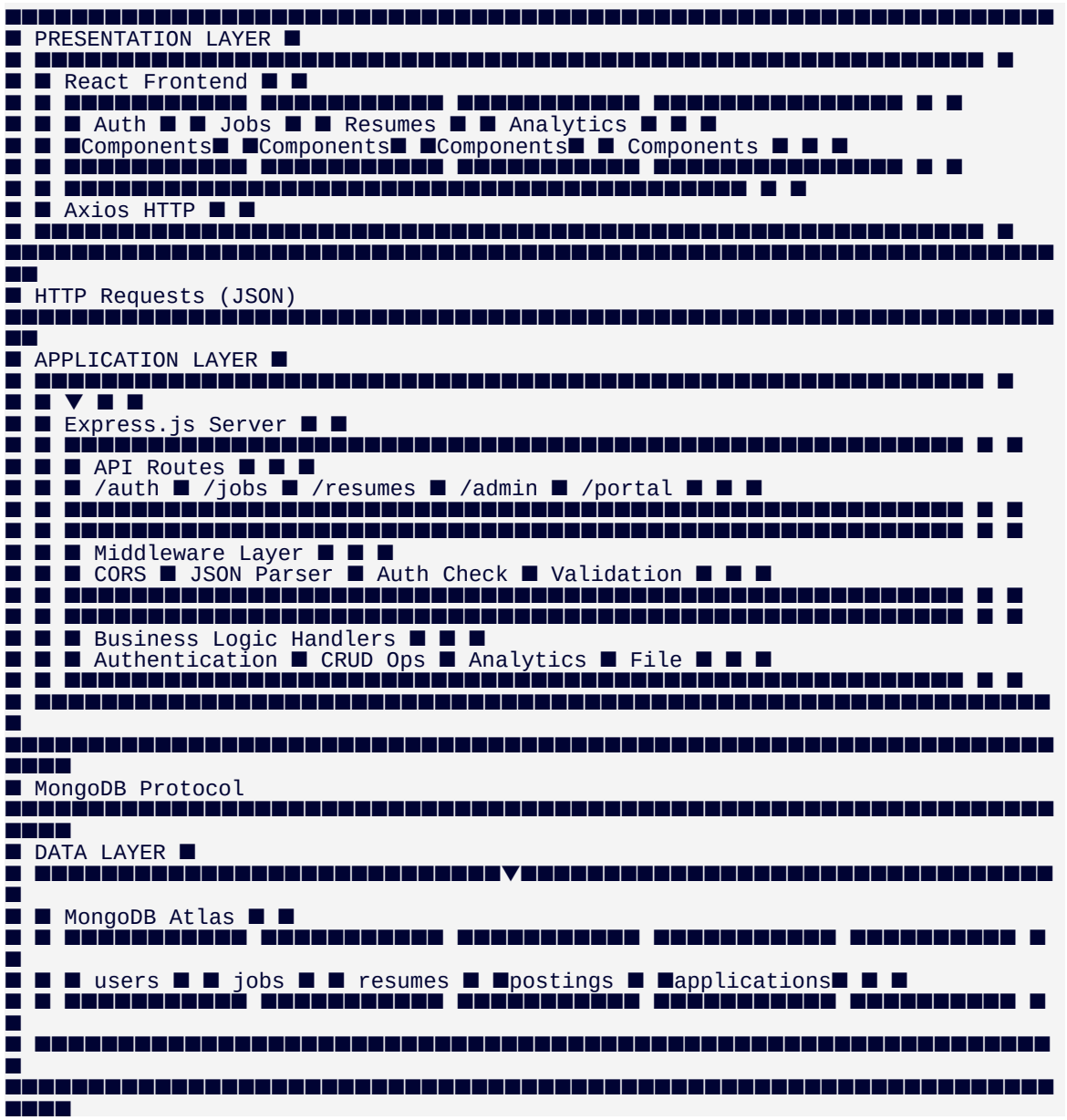
7.1 Architecture Overview

The Job Tracking System follows a **three-tier architecture** consisting of:

Presentation Layer (Frontend): React.js application running in the browser

Application Layer (Backend): Node.js with Express handling business logic

Data Layer (Database): MongoDB Atlas cloud database



7.2 Frontend Architecture

Component Hierarchy

```
App.js (Root Component)
■■■ Routes Configuration
■ ■■■ / → Welcome.js
■ ■■■ /login → Login.js
■ ■■■ /signup → Signup.js
■ ■■■ /dashboard → Dashboard.js
■ ■■■ /admin-dashboard → AdminDashboard.js
■ ■■■ /job-tracker → JobTracker.js
■ ■■■ /resume-builder → ResumeBuilder.js
■ ■■■ /job-portal → JobPortalEntry.js
■ ■■■ /job-listings → JobListingPage.js
■ ■■■ /recruiter-dashboard → RecruiterDashboard.js
■ ■■■ ... (other routes)
■■■ Navbar.js (Navigation)
```

State Management

- **Local State**: React useState for component-level data
- **Global State**: LocalStorage for user session persistence
- **Server State**: API data fetched via Axios, cached in components

Service Layer

```
// services/api.js
API Instance (Axios)
■■■ baseURL: http://127.0.0.1:5000
■■■ Headers: Content-Type: application/json
■■■ Methods: GET, POST, PUT, DELETE, PATCH
```

7.3 Backend Architecture

Server Structure

```
server.js
■■■ Configuration
■ ■■■ dotenv.config()
■ ■■■ express app setup
■ ■■■ CORS middleware
■ ■■■ JSON body parser (10MB limit)
■■■ Database Connection
■ ■■■ MongoClient.connect()
■ ■■■ Database selection (jobtracker)
■ ■■■ Collections initialization
■■■ Middleware Functions
■ ■■■ toObjectId() - ID conversion
■ ■■■ serializeJob() - Data transformation
■ ■■■ serializeResume() - Data transformation
■ ■■■ serializeUser() - Data transformation
■■■ Route Handlers
```

```
■ ■■■ Auth Routes (/signup, /login)
■ ■■■ Job Routes (/jobs, /jobs/:id)
■ ■■■ Resume Routes (/resumes, /resumes/:id)
■ ■■■ Job Portal Routes (/job-postings, /apply-job)
■ ■■■ Admin Routes (/admin/*)
■ ■■■ Utility Routes (/analyze-skills, /reset-db)
■■■ Server Startup
■■■ Index creation
■■■ app.listen(PORT)
```

Request-Response Flow

```
1. Client sends HTTP Request
2. CORS middleware processes headers
3. Body parser middleware parses request data
4. Route handler matches URL and HTTP method
5. Authentication/Authorization checks (if required)
6. Input validation
7. Database operation (CRUD)
8. Response serialization
9. JSON response sent to client
10. Client updates UI based on response
```

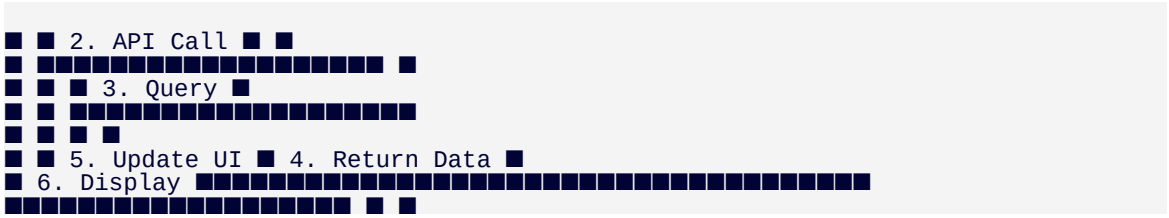
7.4 Database Architecture

Connection Architecture

```
Application → MongoDB Driver → MongoDB Atlas Cluster
■ ■ ■
■ Connection Pool Replica Set
■ (Multiple (3 Nodes)
■ Connections) - Primary
■ - Secondary 1
■ - Secondary 2
■
■■■ Collections:
■■■ users (unique index on username)
■■■ jobs (index on user_id)
■■■ resumes (index on user_id)
■■■ job_postings (index on recruiter_id)
■■■ job_applications (indexes on user_id, job_id)
■■■ job_seeker_resumes (index on user_id)
```

Data Flow Diagram

```
■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■
■ User ■■■■■■■■ React ■■■■■■■■ Express ■■■■■■■■ MongoDB ■
■ Browser ■■■■■■■■ Frontend■■■■■■■■ Backend ■■■■■■■■ Atlas ■
■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■
■ ■ ■ ■
■ 1. User Action ■ ■ ■
■■■■■■■■■■■■■■■■■■■■■ ■ ■
```



8. DETAILED MODULE DESCRIPTION

8.1 Authentication Module

Purpose

Manages user identity, access control, and session management across the platform.

Components

Backend Implementation

```
// User Registration
POST /signup
Request: { username, password, role }
Response: { message: 'User created' }
Validation: Username uniqueness, password presence

// User Login
POST /login
Request: { username, password }
Response: { message, role, user_id }
Error Cases: Invalid credentials (401)
```

Frontend Implementation

- **Signup.js**: Registration form with username, password, role selection
- **Login.js**: Authentication form with credential validation
- **JobSeekerSignup.js**: Specialized signup for job portal
- **RecruiterSignup.js**: Specialized signup for recruiters

• **Session Management**: localStorage stores user_id, role, username

User Roles

client (default): Job seeker with personal dashboard access
admin: System administrator with full access
recruiter: Can post jobs and review applications
jobseeker: Portal-specific job seeker role

Security Features

• Password stored as plaintext (note: should use hashing in production)
• Role validation on protected routes
• User ID validation using ObjectId conversion
• CORS restricted to frontend origin

Workflow

User fills registration form → POST /signup
Server checks username uniqueness → Creates user document
User logs in → POST /login
Server validates credentials → Returns user data
Frontend stores user info in localStorage
Protected routes check role before rendering

8.2 Job Tracking Module

Purpose

Enables users to manage their job application lifecycle from discovery to offer.

Features

Job Application Management

```
// Add Job
POST /jobs
Fields:
- user_id (required): Owner reference
- company (required): Company name
- role (required): Job position
- status (required): Application status
- notes (optional): Additional information
- deadline (optional): Application deadline
- date_applied (optional): Submission date
- followup (optional): Follow-up date
- url (optional): Job posting URL

// Get Jobs
GET /jobs?user_id=xxx&role=client
Admin override: role=admin returns all jobs

// Update Job
PUT /jobs/:jobId
Authorization: Owner or admin only

// Delete Job
DELETE /jobs/:jobId
Authorization: Owner or admin only
```

Notes Management

```
// Get Notes
GET /jobs/:jobId/notes

// Update Notes
POST /jobs/:jobId/notes
Body: { user_id, role, notes }
```

Frontend Components

- **JobTracker.js**: Main job management interface
- **AddJob.js**: Form for adding new applications
- **Dashboard.js**: Overview with job statistics
- **DeadlineCalendar.js**: Calendar visualization of deadlines

Status Tracking

Available statuses for applications:

- Applied
- Interview Scheduled
- Phone Screen

- Technical Interview
- Final Round
- Offer Received
- Rejected
- Withdrawn

Workflow

1. User discovers job opportunity
2. Adds job via AddJob.js form
3. System stores in jobs collection
4. User updates status as progress occurs
5. System tracks deadline dates
6. Calendar component displays upcoming deadlines
7. Analytics show status distribution

8.3 Resume Builder Module

Purpose

Provides a structured interface for creating professional resumes with multiple sections.

Data Structure

```
Resume Document:
{
_id: ObjectId,
user_id: ObjectId,
name: String,
personalInfo: {
fullName: String,
email: String,
phone: String,
location: String,
linkedIn: String,
portfolio: String
},
summary: String,
experience: [
{
title: String,
```

```
company: String,
location: String,
startDate: String,
endDate: String,
current: Boolean,
description: String
}
],
education: [
{
degree: String,
institution: String,
location: String,
graduationDate: String,
gpa: String
}
],
skills: [String],
projects: [
{
name: String,
description: String,
technologies: [String],
link: String
}
],
created_at: Date,
updated_at: Date
}
```

API Endpoints

```
// Create Resume
POST /resumes

// Get User's Resumes
GET /resumes?user_id=xxx&role=client

// Update Resume
PUT /resumes/:resumeId
Authorization: Owner only (compares user_id)

// Delete Resume
DELETE /resumes/:resumeId
Authorization: Owner only
```

Frontend Component

- **ResumeBuilder.js**: Comprehensive form with:
- Personal information section
- Professional summary textarea
- Experience entries (add/remove dynamically)
- Education entries (add/remove dynamically)
- Skills input (comma-separated or array)
- Projects section with technology tags

• Live preview capability
• Save and update functionality

Features

Multiple Resumes: Users can create different versions for different roles
Dynamic Sections: Add unlimited experience, education, and project entries
Auto-save: Updates timestamps on modifications
Validation: Ensures required fields before saving

Workflow

User navigates to Resume Builder
Creates new resume or selects existing
Fills personal information
Adds work experience entries
Adds education history
Lists skills (with suggestions)
Documents projects
Reviews and saves
Can edit or delete anytime

8.4 Job Portal Module

Purpose

Connects job seekers with recruiters through a dedicated marketplace interface.

User Types in Portal

Recruiter Features

```
// Create Job Posting
POST /create-job-posting
Body: {
recruiter_id,
position,
company,
description,
requirements[]
}

// View Recruiter's Jobs
GET /recruiter-jobs/:recruiterId

// Delete Job Posting
DELETE /job-postings/:jobId
Cascades: Deletes all applications for the job
```

Job Seeker Features

```
// View All Job Postings
GET /job-postings

// Apply for Job
POST /apply-job
Body: {
user_id,
job_id,
resume_attachment: { name, data },
cover_page_attachment: { name, data },
referral
}
Validation: Checks for existing application (409 if duplicate)

// View My Applications
GET /job-applications/:userId

// Withdraw Application
DELETE /job-applications/:applicationId
```

Recruiter Application Management

```
// View Applicants for a Job
GET /job-applicants/:jobId
Returns: Array of applications with resume/cover letter

// Update Application Status
PATCH /job-applications/:applicationId/status
Body: { recruiter_id, status }
Status values: 'accepted', 'rejected'
Authorization: Only job's recruiter can update
```

Frontend Components

- **JobPortalEntry.js**: Landing page for portal (seeker/recruiter selection)
- **JobSeekerSignup.js**: Registration for job seekers
- **RecruiterSignup.js**: Registration for recruiters

- **JobListingPage.js**: Browse all available jobs
- **CreateJobPosting.js**: Form for recruiters to post jobs
- **RecruiterDashboard.js**: Manage posted jobs and view applicants
- **JobApplicationsDashboard.js**: Job seekers track their applications
- **ResumeUpload.js**: Upload resume files for applications

Analytics for Recruiters

```
GET /recruiter-analytics/:recruiterId
Returns: {
summary: {
totalJobs,
totalApplicants,
totalAccepted,
totalRejected,
totalPending
},
jobs: [{
job_id,
position,
company,
totalApplicants,
accepted,
rejected,
pending
}]
}
```

Workflow

Recruiter Flow:

Registers as recruiter

Logs in to recruiter dashboard

Creates job posting with requirements

Views list of posted jobs

Clicks on job to see applicants

Reviews resume and cover letter attachments

Updates application status (accept/reject)

Views analytics on hiring metrics

Job Seeker Flow:

Registers as job seeker

Browses job listings

Views job details and requirements

Uploads resume (if not done)

Applies with resume and cover letter

Tracks application status in dashboard

Can withdraw application if needed

8.5 Admin Module

Purpose

Provides system oversight and management capabilities for administrators.

Features

User Management

```
// Get All Clients
GET /admin/clients?role=admin
Returns: All users with role='client' (excludes passwords)

// Delete Client
DELETE /admin/clients/:clientId
Authorization: role=admin required
Cascades: Deletes client's jobs first, then user
```

Job Management

```
// Get All Jobs (Admin View)
GET /admin/jobs?role=admin
Returns: All jobs across all users

// Delete Any Job
DELETE /admin/jobs/:jobId?role=admin
Authorization: Admin can delete any job
```

Database Management

```
// Reset Database (Development Only)
POST /reset-db
Action: Deletes all users and jobs
Warning: Destructive operation
```

Frontend Component

- **AdminDashboard.js**: Comprehensive interface with:
- Statistics cards (total users, jobs, applications)
- Client management table
- Job management table
- Delete confirmations
- Charts showing platform activity

Access Control

- All admin routes check `role === 'admin'`
- Returns 403 Forbidden for non-admin access attempts
- Admin bypasses ownership checks on resources

Workflow

Admin logs in with admin credentials
Dashboard displays platform statistics
Can view all registered clients
Can delete problematic accounts (with cascade)
Can view all job applications
Can remove inappropriate job postings
Has read-only access to all user data

8.6 File Upload Module

Purpose

Handles document uploads for resumes and cover letters in job applications.

Implementation Approach

- **Storage Method**: Base64 encoding in MongoDB
- **File Types**: PDF, DOC, DOCX (client-side validation recommended)
- **Size Limit**: 10MB per file (configured in Express middleware)

API Endpoints

Job Application Attachments

```
POST /apply-job
Body: {
user_id,
job_id,
resume_attachment: {
name: "resume.pdf",
data: "base64encodedstring..."
},
cover_page_attachment: {
name: "cover_letter.pdf",
data: "base64encodedstring..."
}
}
Validation:
- Checks file name and data presence
- Validates user and job existence
- Prevents duplicate applications
```

Job Seeker Resume Uploads

```
// Upload Resume File
POST /upload-resume
Body: {
user_id,
resume_name,
resume_file: "base64encodedcontent..."
}

// Get User's Uploaded Resumes
GET /resumes/:userId
Returns: Array of resume documents with file_content
```

Frontend Components

- **ResumeUpload.js**: Interface for uploading and managing resume files
- **JobListingPage.js**: Application form with file selection
- **RecruiterDashboard.js**: Displays attachments for review

File Handling Process

User selects file via HTML file input

JavaScript FileReader converts to Base64

Base64 string sent in JSON payload

Server stores in MongoDB document

Retrieval decodes Base64 for display/download

Storage Structure

```
job_applications collection:
{
resume_attachment: {
name: String, // Original filename
data: String // Base64 encoded content
},
cover_page_attachment: {
name: String,
data: String
}
}

job_seeker_resumes collection:
{
user_id: ObjectId,
name: String,
file_content: String // Base64 encoded
}
```

Security Considerations

- File size limits prevent database bloat
- Base64 encoding increases size by ~33%
- No virus scanning (recommended for production)
- File type validation should be client-side

8.7 Skill Analysis Module

Purpose

Automatically extracts relevant skills from job descriptions to help job seekers identify requirements.

Implementation

- **Algorithm**: Keyword matching against predefined skill database
- **Processing**: Case-insensitive text matching with normalization
- **Output**: Array of matching skills with count

API Endpoint

```
POST /analyze-skills
Request: { description: String }
Response: {
skills: [String], // Up to 12 unique skills
count: Number
}

Skill Database (40+ skills):
- Programming: JavaScript, Python, Java, TypeScript, React, Node.js, Angular
- Databases: MongoDB, SQL, PostgreSQL
- Cloud: AWS, Azure, Google Cloud, Docker, Kubernetes
- Soft Skills: Communication, Leadership, Teamwork, Problem Solving
- Practices: Agile, Scrum, DevOps, CI/CD, Testing
- Design: UI/UX Design, Figma, Adobe Creative Suite
```

Contextual Analysis

The system adds inferred skills based on keywords:

```
if (text.includes("senior") || text.includes("lead")) {
skills.push("Leadership", "Mentoring");
}
if (text.includes("remote")) {
skills.push("Remote Collaboration", "Time Management");
}
if (text.includes("team")) {
skills.push("Teamwork", "Collaboration");
}
```

Frontend Component

- **SkillSuggestor.js**: Interface for skill analysis
- Textarea for job description input
- Analyze button triggers API call
- Displays extracted skills as tags
- Helps users identify skill gaps

Workflow

User copies job description
Pastes into SkillSuggestor textarea
Clicks "Analyze Skills" button
Frontend sends POST /analyze-skills
Backend processes text against skill database
Returns matching skills array
Frontend displays as clickable tags
User can add identified skills to their profile

Limitations

• Keyword-based (not NLP/AI powered)
• Limited to predefined skill list
• May miss industry-specific or emerging skills
• Contextual analysis is rule-based

9. API DESIGN

9.1 API Overview

The Job Tracking System implements a RESTful API with the following characteristics:

• **Base URL**: `http://localhost:5000` (development)
• **Data Format**: JSON for both requests and responses
• **Authentication**: Role-based via request parameters
• **Status Codes**: Standard HTTP status codes

9.2 Authentication Endpoints

POST /signup

Purpose: Register new user account

Request Body:

```
{
"username": "john_doe",
"password": "securepass123",
"role": "client"
}
```

Success Response (200):

```
{
"message": "User created"
}
```

Error Responses:

- 400: `{"error": "Username and password are required"}`
- 400: `{"error": "User already exists"}` (duplicate username)
- 500: `{"error": "Error message"}` (server error)

POST /login

Purpose: Authenticate existing user

Request Body:

```
{
"username": "john_doe",
"password": "securepass123"
}
```

Success Response (200):

```
{
"message": "Login successful",
"role": "client",
"user_id": "507f1f77bcf86cd799439011"
}
```

Error Response:

- 401: `{"error": "Invalid credentials"}`

9.3 Job Management Endpoints

POST /jobs

Purpose: Add new job application

Request Body:

```
{
"user_id": "507f1f77bcf86cd799439011",
"company": "Google",
"role": "Software Engineer",
"status": "Applied",
"notes": "Applied through referral",
"deadline": "2024-12-31",
"date_applied": "2024-01-15",
"followup": "2024-01-22",
"url": "https://careers.google.com"
}
```

Success Response (200):

```
{
"message": "Job added",
"id": "507f1f77bcf86cd799439012"
}
```

GET /jobs

Purpose: Retrieve job applications

Query Parameters:

- `user_id` (required): Filter by user
- `role` (optional): 'admin' returns all jobs

Success Response (200):

```
[
{
"id": "507f1f77bcf86cd799439012",
"user_id": "507f1f77bcf86cd799439011",
"company": "Google",
"role": "Software Engineer",
```

```
"status": "Applied",
"notes": "",
"deadline": "2024-12-31",
"date_applied": "2024-01-15",
"followup": "2024-01-22",
"url": "https://careers.google.com"
}
]
```

PUT /jobs/:jobId

Purpose: Update existing job

Request Body:

```
{
"user_id": "507f1f77bcf86cd799439011",
"user_role": "client",
"company": "Google",
"job_role": "Senior Software Engineer",
"status": "Interview Scheduled",
"deadline": "2024-12-31",
"date_applied": "2024-01-15",
"followup": "2024-01-22",
"notes": "Phone screen scheduled",
"url": "https://careers.google.com"
}
```

Authorization: Owner or admin only

DELETE /jobs/:jobId

Purpose: Delete job application

Query Parameters:

- `user_id` (required)
- `role` (optional): 'admin' bypasses ownership

Success Response (200):

```
{
"message": "Deleted"
}
```

9.4 Resume Management Endpoints

POST /resumes

Purpose: Create new resume

Request Body:

```
{
"user_id": "507f1f77bcf86cd799439011",
"name": "Software Developer Resume",
"personalInfo": {
"fullName": "John Doe",
"email": "john@example.com",
"phone": "+1-555-0123",
"location": "San Francisco, CA"
},
"summary": "Experienced developer...",
"experience": [...],
"education": [...],
"skills": ["JavaScript", "React", "Node.js"],
"projects": [...]
}
```

GET /resumes

Purpose: Retrieve user's resumes

Query Parameters:

- `user_id` (required)
- `role` (optional)

PUT /resumes/:resumeId

Purpose: Update resume

Authorization: Owner only

DELETE /resumes/:resumeId

Purpose: Delete resume

Query Parameters:

- `user_id` (required)

9.5 Job Portal Endpoints

POST /create-job-posting

Purpose: Recruiter creates job listing

Request Body:

```
{
"recruiter_id": "507f1f77bcf86cd799439011",
"position": "Full Stack Developer",
"company": "TechCorp",
"description": "We are looking for...",
"requirements": ["3+ years experience", "JavaScript", "React"]
}
```

GET /job-postings

Purpose: List all available jobs

Response: Array of job postings sorted by created_at (descending)

POST /apply-job

Purpose: Job seeker applies for position

Request Body:

```
{
"user_id": "507f1f77bcf86cd799439011",
"job_id": "507f1f77bcf86cd799439012",
"resume_attachment": {
"name": "resume.pdf",
"data": "base64encodeddata..."
},
"cover_page_attachment": {
```

```
"name": "cover.pdf",
"data": "base64encodeddata..."
},
"referral": "Employee referral code"
}
```

Error Response:

- 409: Already applied for this job

PATCH /job-applications/:applicationId/status

Purpose: Recruiter updates application decision

Request Body:

```
{
"recruiter_id": "507f1f77bcf86cd799439011",
"status": "accepted"
}
```

Valid Status: 'accepted', 'rejected'

GET /recruiter-analytics/:recruiterId

Purpose: Get hiring metrics

Response:

```
{
"summary": {
"totalJobs": 5,
"totalApplicants": 45,
"totalAccepted": 3,
"totalRejected": 12,
"totalPending": 30
},
"jobs": [...]
}
```

9.6 Admin Endpoints

GET /admin/clients

Purpose: List all client users

Query Parameters:

- `role=admin` (required)

DELETE /admin/clients/:clientId

Purpose: Delete client account

Action: Deletes client's jobs, then deletes client

GET /admin/jobs

Purpose: List all jobs in system

DELETE /admin/jobs/:jobId

Purpose: Delete any job

POST /reset-db

Purpose: Clear database (development only)

9.7 Utility Endpoints

POST /analyze-skills

Purpose: Extract skills from job description

Request Body:

```
{
"description": "Looking for React developer with Node.js experience..."
}
```

Response:

```
{
"skills": ["React", "Node.js", "JavaScript"],
"count": 3
}
```

9.8 Error Response Format

All errors follow consistent format:

```
{
"error": "Descriptive error message"
}
```

Common Status Codes:

- 200: Success
- 201: Created
- 400: Bad Request (validation error)
- 401: Unauthorized (invalid credentials)
- 403: Forbidden (insufficient permissions)
- 404: Not Found
- 409: Conflict (duplicate resource)
- 500: Internal Server Error

10. DATABASE DESIGN

10.1 Database Overview

Database Name: `jobtracker`

Type: MongoDB (NoSQL Document Database)

Hosting: MongoDB Atlas (Cloud)

Driver: MongoDB Node.js Driver v6.21.0

10.2 Collection: users

Purpose: Store user authentication and role information

Schema:

```
{
_id: ObjectId, // Auto-generated unique identifier
username: String, // Unique login identifier
password: String, // User password (should hash in production)
role: String // 'client', 'admin', 'recruiter', 'jobseeker'
}
```

Indexes:

- `{ username: 1 }` - Unique index for fast lookup and duplicate prevention

Sample Document:

```
{
"_id": "507f1f77bcf86cd799439011",
"username": "john_doe",
"password": "securepass123",
"role": "client"
}
```

10.3 Collection: jobs

Purpose: Store job application records for users

Schema:

```
{
_id: ObjectId,
user_id: ObjectId, // Reference to users collection
company: String, // Company name
role: String, // Job title/position
status: String, // Application status
notes: String, // Additional notes
deadline: String, // ISO date or formatted string
date_applied: String, // Application date
followup: String, // Follow-up date
url: String // Job posting URL
}
```

Indexes:

- `{ user_id: 1 }` - For fast user-specific job queries

Sample Document:

```
{
"_id": "507f1f77bcf86cd799439012",
"user_id": "507f1f77bcf86cd799439011",
"company": "Google",
"role": "Software Engineer",
"status": "Interview Scheduled",
"notes": "Phone screen on Friday",
"deadline": "2024-12-31",
"date_applied": "2024-01-15",
"followup": "2024-01-22",
"url": "https://careers.google.com"
}
```

10.4 Collection: resumes

Purpose: Store resume builder data

Schema:

```
{
_id: ObjectId,
user_id: ObjectId,
name: String,
personalInfo: {
fullName: String,
email: String,
phone: String,
location: String,
linkedIn: String,
```

```
portfolio: String
},
summary: String,
experience: [{
title: String,
company: String,
location: String,
startDate: String,
endDate: String,
current: Boolean,
description: String
}],
education: [{
degree: String,
institution: String,
location: String,
graduationDate: String,
gpa: String
}],
skills: [String],
projects: [{
name: String,
description: String,
technologies: [String],
link: String
}],
created_at: Date,
updated_at: Date
}
```

Indexes:

- `{ user_id: 1 }` - For user-specific resume queries

10.5 Collection: job_postings

Purpose: Store recruiter job listings for the job portal

Schema:

```
{
_id: ObjectId,
recruiter_id: ObjectId, // Reference to users (recruiter)
position: String, // Job title
company: String, // Company name
description: String, // Full job description
requirements: [String], // Array of requirements
created_at: Date,
updated_at: Date
}
```

Indexes:

- `{ recruiter_id: 1 }` - For recruiter's job listings

10.6 Collection: job_applications

Purpose: Store job applications from seekers to recruiter postings

Schema:

```
{
_id: ObjectId,
user_id: ObjectId, // Job seeker
job_id: ObjectId, // Reference to job_postings
recruiter_id: ObjectId, // Job poster
job_title: String,
company_name: String,
user_name: String,
resume_attachment: {
name: String,
data: String // Base64 encoded
},
cover_page_attachment: {
name: String,
data: String // Base64 encoded
},
referral: String,
status: String, // 'pending', 'accepted', 'rejected'
applied_at: Date,
updated_at: Date
}
```

Indexes:

- `{ user_id: 1 }` - For user's applications
- `{ job_id: 1 }` - For job's applicants

10.7 Collection: job_seeker_resumes

Purpose: Store uploaded resume files for job seekers

Schema:

```
{
_id: ObjectId,
user_id: ObjectId,
name: String, // Resume filename
file_content: String, // Base64 encoded file
created_at: Date,
updated_at: Date
}
```

Indexes:

- `{ user_id: 1 }` - For user's uploaded resumes

10.8 Indexing Strategy Summary

| Collection | Index | Type | Purpose |

|---|---|---|---|

| users | username | Unique | Fast login, prevent duplicates |

| jobs | user_id | Standard | User's job queries |

| resumes | user_id | Standard | User's resume queries |

| job_postings | recruiter_id | Standard | Recruiter's job listings |

| job_applications | user_id | Standard | User's applications |

| job_applications | job_id | Standard | Job's applicants |

| job_seeker_resumes | user_id | Standard | User's file queries |

10.9 Data Relationships

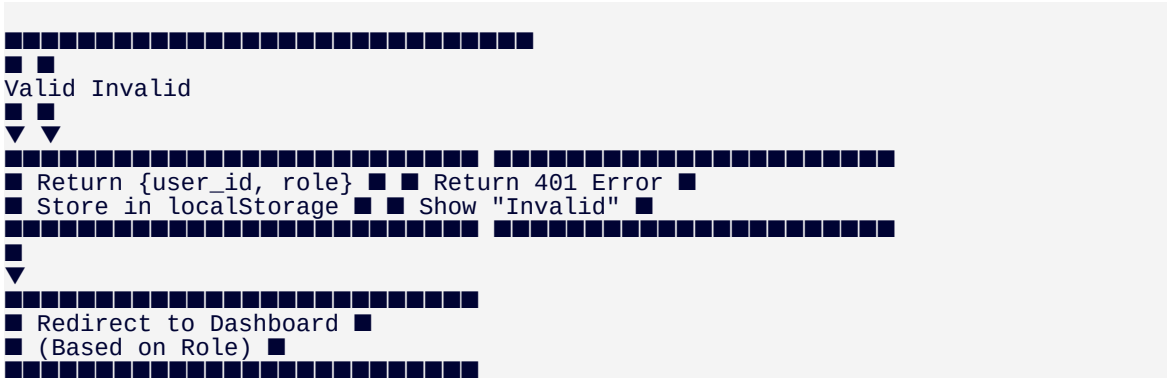
```
users (1) ■■■■■■■■■< (N) jobs
users (1) ■■■■■■■■■< (N) resumes
users (1) ■■■■■■■■■< (N) job_postings (recruiters)
users (1) ■■■■■■■■■< (N) job_applications (seekers)
users (1) ■■■■■■■■■< (N) job_seeker_resumes

job_postings (1) ■■■■■■■■■< (N) job_applications
```

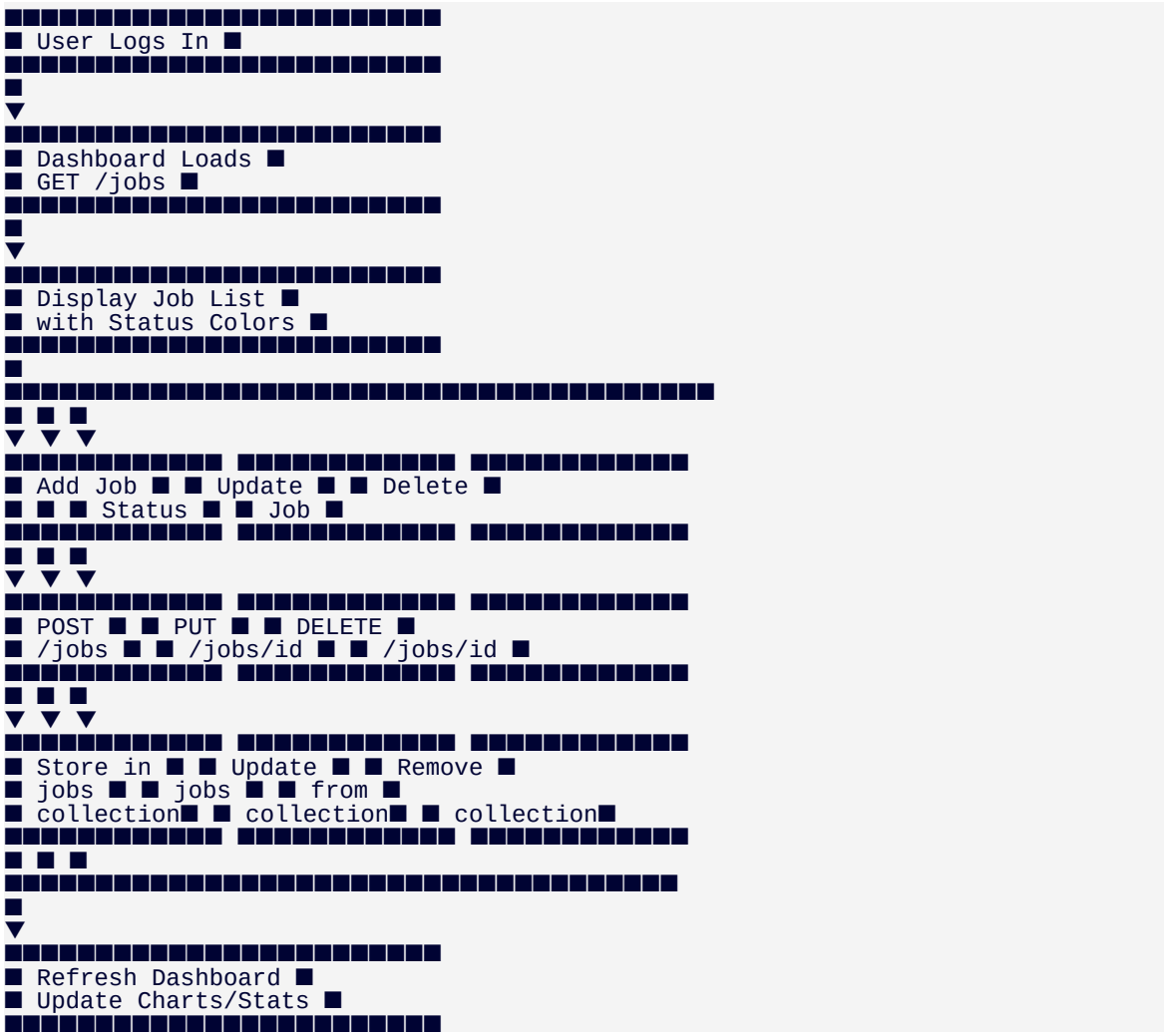
11. SYSTEM WORKFLOW

11.1 User Registration and Login Flow

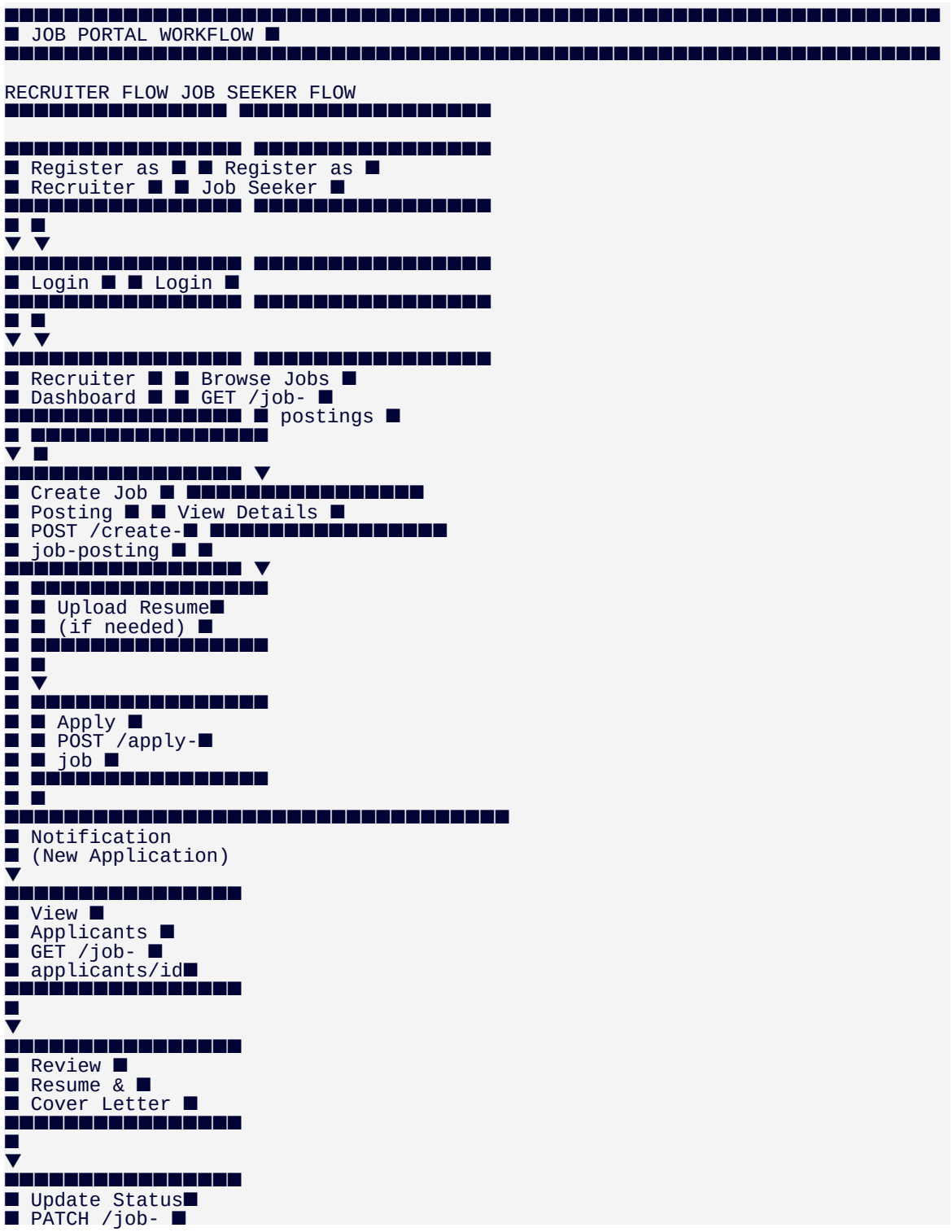


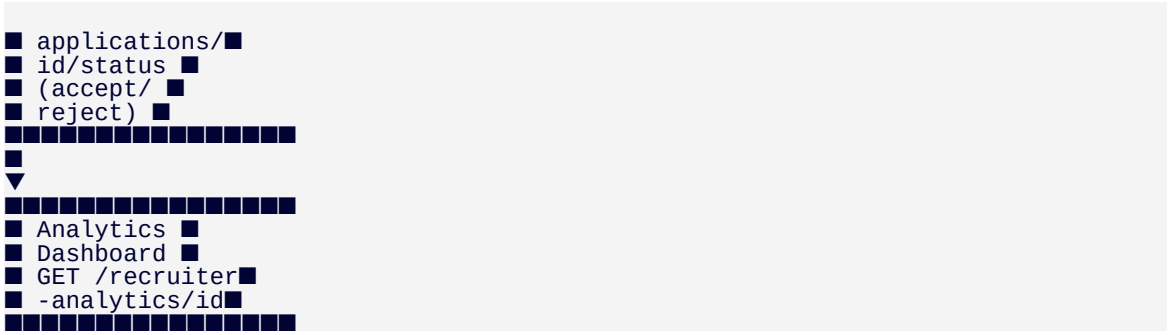


11.2 Job Application Tracking Flow

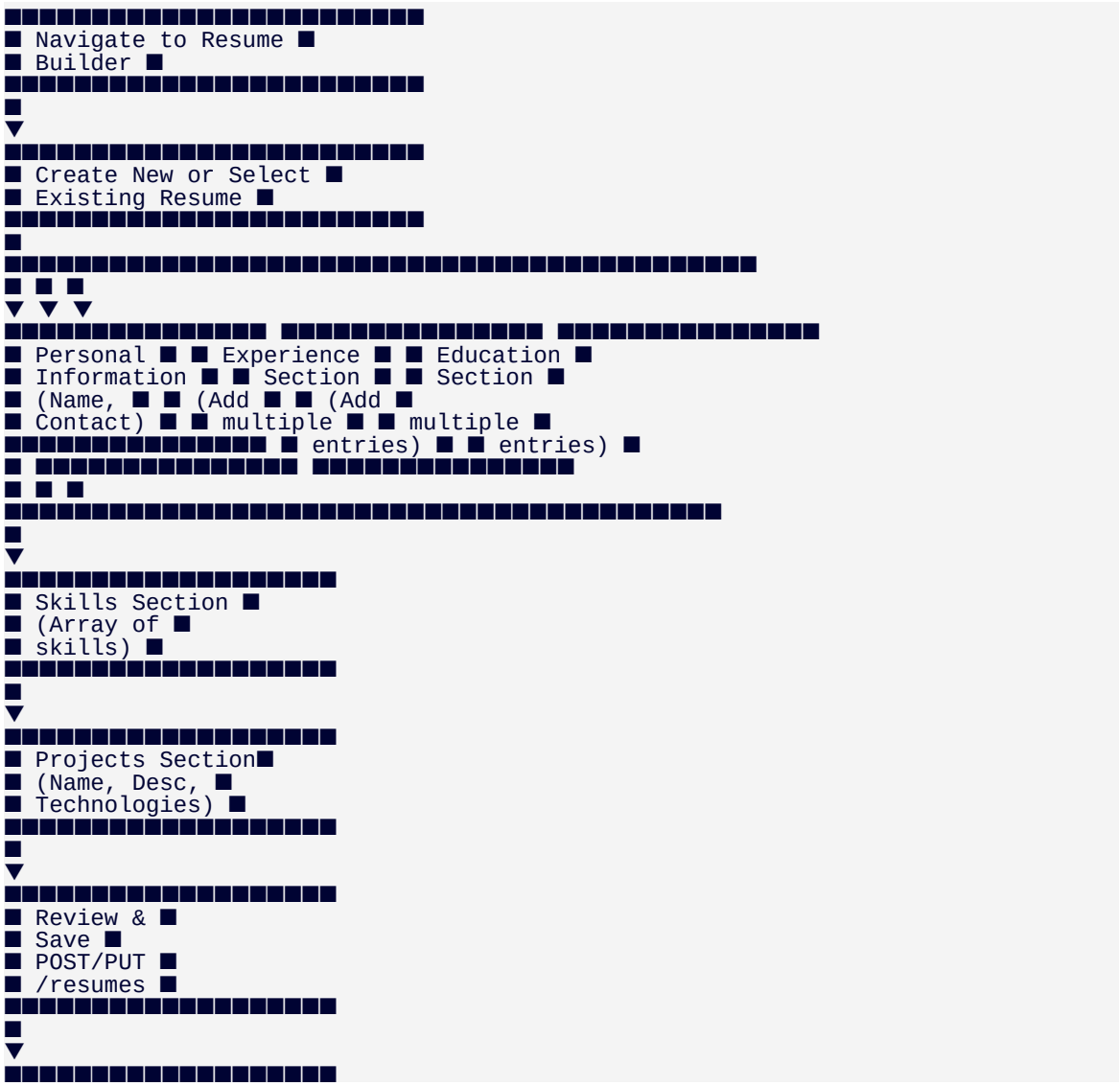


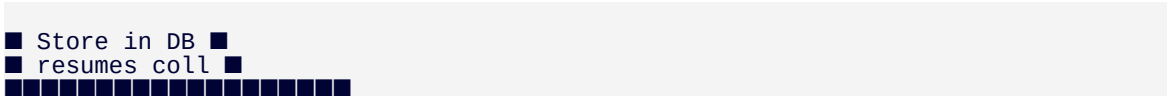
11.3 Job Portal Application Flow





11.4 Resume Builder Flow





12. FEATURES

12.1 Dashboard Features

Personal Dashboard (Job Seeker)

- **Overview Cards**: Total applications, interviews scheduled, offers received, pending applications
- **Status Distribution Chart**: Pie chart showing application status breakdown
- **Timeline View**: Calendar displaying upcoming deadlines and interviews
- **Recent Activity**: Latest updates on job applications
- **Quick Actions**: Buttons to add job, search companies, build resume

Admin Dashboard

- **Platform Statistics**: Total users, total jobs, total applications
- **User Management Table**: List all clients with delete option
- **Job Management Table**: View all jobs with admin delete capability
- **Activity Charts**: Visual representation of platform usage

Recruiter Dashboard

- **Posted Jobs List**: All jobs created by the recruiter
- **Applicant Counts**: Number of applicants per job
- **Quick Stats**: Total applicants, acceptance rate
- **Action Buttons**: View applicants, delete posting, create new job

12.2 Resume Builder Features

Section Management

Personal Information

• Full name, email, phone

• Location, LinkedIn, Portfolio URL

Professional Summary

• Free-form text area

• Suggested length indicators

Work Experience

• Add unlimited entries

• Each entry: Title, Company, Location, Dates, Description

• "Current" checkbox for ongoing positions

Education

• Add unlimited entries

• Each entry: Degree, Institution, Location, Graduation Date, GPA

Skills

• Tag-based input

• Suggestions based on industry

• Remove individual skills

Projects

• Project name and description

• Technologies used (array)

• Project link URL

Features

• **Auto-save**: Timestamps track creation and updates

• **Multiple Resumes**: Create different versions for different roles

• **Live Preview**: See changes as you type

- **Validation**: Required field indicators

12.3 Job Portal Features

For Job Seekers

- **Job Listings Page**: Browse all available positions
- **Search & Filter**: Filter by company, position, requirements
- **Job Details View**: Full description and requirements
- **One-Click Apply**: Apply with stored resume
- **Application Tracking**: View all submitted applications
- **Status Monitoring**: Track application progress (pending/accepted/rejected)
- **Withdraw Option**: Remove application if needed

For Recruiters

- **Job Posting Form**: Create detailed job listings
- **Requirements Builder**: Add multiple requirements as tags
- **Applicant Review**: View all candidates with documents
- **Document Viewer**: Access resume and cover letter attachments
- **Status Updates**: Accept or reject applicants
- **Analytics Dashboard**: View statistics on each job posting

12.4 Analytics Features

Visual Charts (Chart.js & Recharts)

Application Status Pie Chart: Distribution of statuses

Monthly Applications Bar Chart: Applications over time

Response Rate Metrics: Conversion funnel visualization

Company-wise Distribution: Jobs by company

Recruiter Analytics

- **Total Jobs Posted**: Count of active listings
- **Total Applicants**: Overall candidate count
- **Conversion Rates**: Accepted vs Rejected vs Pending
- **Per-Job Statistics**: Applicant count per position

12.5 Utility Features

Skill Analysis

- **Job Description Parser**: Paste description, extract skills
- **Skill Suggestions**: Shows matching skills as tags
- **Gap Analysis**: Compare extracted skills with user's profile

Deadline Calendar

- **Visual Calendar**: Month/Week views
- **Deadline Indicators**: Color-coded by urgency
- **Event Details**: Click to see job details
- **Reminder Visualization**: Upcoming deadlines highlighted

Company Search

- **Research Tool**: Search for company information
- **Notes Integration**: Link research to job applications

File Management

- **Resume Upload**: Store multiple resume versions
- **Cover Letter Upload**: Attach to applications
- **File Browser**: Select from previously uploaded files

13. ADVANTAGES

13.1 For Job Seekers

Centralized Organization

• All job applications in one place

• No more scattered spreadsheets or notes

• Easy access from any device

Visual Progress Tracking

• Color-coded status indicators

• Charts showing application pipeline

• Calendar view for deadlines

Professional Resume Building

• Structured format ensures completeness

• Multiple versions for different roles

• Consistent formatting

Time Management

• Deadline reminders prevent missed opportunities

• Follow-up date tracking

• Historical data for analysis

Skill Development Insights

• Skill extraction identifies requirements

• Gap analysis for self-improvement

• Industry trend awareness

13.2 For Recruiters

Efficient Candidate Management

• Centralized applicant database

• Quick status updates

• Document storage and retrieval

Hiring Pipeline Visibility

• Analytics show conversion rates
• Identify bottlenecks in process
• Data-driven hiring decisions

Professional Branding

• Structured job postings
• Consistent communication channel
• Organized application process

Cost and Time Savings

• Reduced administrative overhead
• Automated status tracking
• Quick applicant screening

13.3 For Administrators

Platform Oversight

• Monitor user activity
• Identify and remove inappropriate content
• Ensure system integrity

User Management

• Easy account administration
• Bulk operations capability
• Role-based access control

13.4 Technical Advantages

Modern Technology Stack

• React ensures smooth user experience
• Node.js provides fast response times
• MongoDB offers flexible data storage

Scalable Architecture

• Component-based frontend allows easy■■

• RESTful API supports multiple clients

• Cloud database handles growth

Cross-Platform Accessibility

• Web-based works on any device

• Responsive design for mobile

• No installation required

Security Features

• Role-based access control

• Input validation

• CORS protection

14. LIMITATIONS

14.1 Current System Limitations

Security Limitations

Password Storage

• Passwords stored as plaintext

• No hashing or encryption implemented

• Should use bcrypt or similar in production

Authentication

• No JWT or session tokens

• Relies on localStorage which is vulnerable to XSS

• No password reset functionality

Authorization

- Simple role checking
- No granular permissions
- No API rate limiting

Feature Limitations

No Email Notifications

- No automated emails for deadlines
- No status change notifications
- No weekly summary reports

No Real-time Updates

- Requires page refresh for updates
- No WebSocket integration
- No push notifications

File Handling

- Base64 encoding increases file size by 33%
- 10MB limit may be restrictive
- No virus scanning
- No file type validation on server

Skill Analysis

- Keyword-based only (no AI/ML)
- Limited to predefined skill list
- No semantic understanding

Scalability Limitations

Database Design

- No sharding configuration
- Base64 files stored in database (not recommended for large scale)
- No caching layer (Redis)

API Design

- No pagination on list endpoints
- Could become slow with large datasets

• No query optimization

User Experience Limitations

No Offline Support

• Requires constant internet connection
• No Progressive Web App (PWA) features
• No data sync when reconnecting

Limited Customization

• Fixed color schemes
• No theme options
• Limited dashboard widget configuration

No Collaboration

• No sharing features
• No mentor/counselor access
• No team job tracking

Mobile Experience

Responsive but not Native

• Mobile browser only
• No native app features
• Touch interactions not optimized

15. FUTURE ENHANCEMENTS

15.1 Security Improvements

Authentication Overhaul

• Implement JWT-based authentication

• Add refresh token mechanism
• Enable OAuth (Google, LinkedIn login)
• Add two-factor authentication (2FA)

Password Security
• Implement bcrypt password hashing
• Add password strength requirements
• Enable password reset via email
• Add brute force protection

Data Protection
• Encrypt sensitive data at rest
• Implement HTTPS-only cookies
• Add CSRF protection
• Regular security audits

15.2 Feature Additions

Communication Features

Email Integration
• Automated deadline reminders
• Status change notifications
• Weekly progress summaries
• Interview scheduling emails

In-App Messaging
• Chat between job seekers and recruiters
• Interview scheduling interface
• Group discussions for teams

Calendar Integration
• Sync with Google Calendar
• Outlook calendar integration
• Automatic interview scheduling

AI and Machine Learning

Smart Matching

- AI-powered job recommendations
- Resume-job matching score
- Skill gap analysis with learning resources

Natural Language Processing

- Advanced skill extraction from descriptions
- Resume parsing from uploaded documents
- Sentiment analysis on application feedback

Predictive Analytics

- Success probability predictions
- Optimal application timing suggestions
- Salary range predictions

External Integrations

Job Board APIs

- LinkedIn Jobs integration
- Indeed API connection
- Glassdoor data enrichment

Professional Networks

- LinkedIn profile import
- GitHub portfolio integration
- Stack Overflow reputation display

Assessment Tools

- Coding challenge platforms (HackerRank, LeetCode)
- Personality assessment integration
- Skills testing platforms

Enhanced Resume Features

Templates

- Multiple professional templates

• Industry-specific formats
• ATS-friendly layouts

Export Options
• PDF generation
• Word document export
• Shareable links

Resume Analysis
• ATS compatibility scoring
• Improvement suggestions
• Keyword optimization tips

15.3 Technical Enhancements

Performance Improvements

Caching Layer
• Redis for session management
• API response caching
• CDN for static assets

Database Optimization
• Database sharding for scale
• Read replicas for query distribution
• Archive old data automatically

Frontend Optimization
• Code splitting and lazy loading
• Service Workers for offline support
• Image optimization and WebP format

Architecture Improvements

Microservices
• Split into domain-specific services

• Separate authentication service
• Independent analytics service

Real-time Features
• WebSocket integration
• Server-Sent Events for updates
• Push notification service

Mobile Development
• React Native mobile app
• Native push notifications
• Biometric authentication

15.4 Business Features

Premium Tiers
• Advanced analytics for premium users
• Priority application visibility
• Direct recruiter messaging

Enterprise Solutions
• University career center dashboards
• Corporate recruitment management
• API access for integrations

Marketplace Features
• Resume review services
• Career coaching marketplace
• Skill certification integration

16. CONCLUSION

The **Job Tracking System** represents a comprehensive solution to the challenges faced by job seekers, recruiters, and administrators in the modern employment landscape. By leveraging modern web technologies including React.js, Node.js with Express, and MongoDB Atlas, the system delivers a robust, scalable, and user-friendly platform for managing the entire job application lifecycle.

16.1 Project Achievements

This project successfully implements:

Multi-Role Architecture: Distinct interfaces and functionalities for job seekers, recruiters, and administrators, each optimized for their specific needs.

Comprehensive Job Management: Complete CRUD operations for job applications with status tracking, deadline management, and notes functionality.

Professional Resume Builder: A structured, section-based resume creation tool that ensures users create complete, professional resumes.

Integrated Job Portal: A marketplace connecting job seekers with recruiters, complete with application submission, document upload, and status tracking.

Data Visualization: Analytics dashboards using Chart.js and Recharts that provide insights into application progress and hiring metrics.

Skill Analysis: An automated skill extraction feature that helps job seekers identify requirements from job descriptions.

16.2 Technical Merit

The technical implementation demonstrates:

• **Full-Stack Proficiency**: Integration of frontend, backend, and database layers into a cohesive application

• **RESTful API Design**: Well-structured, documented API endpoints following REST conventions

• **NoSQL Database Design**: Effective use of MongoDB's document-oriented structure for flexible data storage

• **Security Awareness**: Implementation of role-based access control, input validation, and CORS protection

• **Modern Development Practices**: Component-based architecture, version control, and environment management

16.3 Real-World Impact

The system addresses genuine pain points in the job search process:

• **Organization**: Replaces disorganized spreadsheets and notes with a centralized platform

• **Efficiency**: Automates tracking and provides visual progress indicators

• **Professionalism**: Enables creation of polished resumes and streamlined application submissions

• **Connectivity**: Facilitates direct connection between job seekers and recruiters

• **Insights**: Provides data-driven understanding of job search effectiveness

16.4 Learning Outcomes

This project provided practical experience in:

Full-stack web application development

NoSQL database design and optimization

RESTful API architecture and implementation

React component architecture and state management

User authentication and authorization systems

Data visualization and analytics presentation

File handling and storage strategies

Role-based access control implementation

16.5 Final Remarks

The Job Tracking System successfully demonstrates how modern web technologies can be applied to solve real-world problems. While there are areas for future enhancement, particularly in security

hardening and advanced features like AI-powered matching, the current implementation provides a solid foundation that effectively meets its stated objectives.

The modular architecture ensures that future enhancements can be integrated without disrupting existing functionality. As the job market continues to evolve, platforms like this will play an increasingly important role in helping individuals manage their careers and assisting organizations in finding the right talent.

This project serves as both a practical tool for job management and a testament to the power of full-stack development in creating solutions that make a meaningful difference in users' professional lives.

APPENDIX A: Project Structure

```
job-tracker-project/
■■■ backend/
■ ■■■ server.js # Express server with all routes
■ ■■■ package.json # Backend dependencies
■ ■■■ package-lock.json # Locked dependency versions
■ ■■■ .env # Environment variables (MongoDB URI, PORT)
■■■ frontend/
■ ■■■ src/
■ ■ ■■■ App.js # Root component with routes
■ ■ ■■■ App.css # Global styles
■ ■ ■■■ index.js # Application entry point
■ ■ ■■■ index.css # Base styles
■ ■ ■■■ services/
■ ■ ■ ■■■ api.js # Axios API instance
■ ■ ■■■ components/
■ ■ ■■■ Welcome.js
■ ■ ■■■ Login.js
■ ■ ■■■ Signup.js
■ ■ ■■■ Dashboard.js
■ ■ ■■■ AdminDashboard.js
■ ■ ■■■ AddJob.js
■ ■ ■■■ JobTracker.js
■ ■ ■■■ ResumeBuilder.js
■ ■ ■■■ SearchCompany.js
■ ■ ■■■ DeadlineCalendar.js
■ ■ ■■■ JobPortalEntry.js
■ ■ ■■■ JobSeekerSignup.js
■ ■ ■■■ RecruiterSignup.js
■ ■ ■■■ JobListingPage.js
■ ■ ■■■ JobApplicationsDashboard.js
■ ■ ■■■ ResumeUpload.js
■ ■ ■■■ CreateJobPosting.js
■ ■ ■■■ RecruiterDashboard.js
■ ■ ■■■ RecruiterAnalyticsDashboard.js
■ ■ ■■■ SkillSuggestor.js
■ ■ ■■■ Navbar.js
■ ■ ■■■ Home.js
■ ■ ■■■ JobList.js
```

```
■ ■■■ public/
■ ■■■ package.json # Frontend dependencies
■ ■■■ .env # Frontend environment variables
■■■ package.json # Root package with concurrent scripts
■■■ README.md # Basic project documentation
```

APPENDIX B: Running the Application

Prerequisites

- Node.js (v14 or higher)
- npm (v6 or higher)
- MongoDB Atlas account (or local MongoDB)

Installation Steps

Clone Repository

`bash

git clone <repository-url>

cd job-tracker-project

`

Install Dependencies

`bash

npm run install-all

`

This installs root, backend, and frontend dependencies.

Configure Environment

- Update `backend/.env` with your MongoDB URI
- Set `PORT` (default: 5000)

• Set `DB_NAME` (default: jobtracker)

Start Application

`bash

npm start

`

This starts both backend (port 5000) and frontend (port 3000).

Development Mode

Backend only:

```
npm run backend:dev
```

Uses nodemon for auto-restart on changes.

Frontend only:

```
npm run frontend
```

Document End

Project Documentation prepared for academic submission

Date: April 2026